

F-260701-DIS-420-FPGA_EXPLEO

TP1 *Full adder*

Notations :

- Les expressions en minuscule italique représentent de l'arithmétique binaire, par exemple $x + y$ représente l'addition binaire de x et y , notamment $1 + 1 = 0$ (l'opérateur $+$ correspond à une porte XOR)
- Les expressions en majuscule représentent de l'algèbre booléenne, par exemple $A + B$ représente la proposition A ou B (l'opérateur $+$ correspond à porte OR).

1. Écrivez sous forme d'équation logique le schéma du *full adder*

Un *Full adder* additionne deux bits x et y ainsi que l'éventuelle retenue provenant d'une addition précédente Cin (*carry-in*), et produit :

- le bit de somme S , résultat de l'addition binaire de x , y , et Cin
- un bit de retenue $Cout$ (*carry-out*), si $x = y = 1$, ou si $x + y = 1$ et $Cin = 1$

Soient A et B les bits à additionner, Cin l'éventuelle retenue en entrée, S le bit de somme, et $Cout$ l'éventuelle retenue produite.

Bit de somme

A	B	Cin	S
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

$$S = (A \oplus B) \oplus Cin = A \oplus B \oplus Cin$$

Carry-out bit

A	B	Cin	Cout
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$Cout = AB + (A \oplus B)Cin$$

En substituant $A\bar{B} + B\bar{A}$ pour $A \oplus B$, on obtient :

$$Cout = AB + (A\bar{B} + B\bar{A})Cin = AB + A\bar{B}Cin + B\bar{A}Cin$$

En remarquant que $ACin$ est vrai dès que A et Cin sont vrais quelque soit B , de même que $BCin$ est vrai dès que B et Cin sont vrais quelque soit A , on obtient :

$$Cout = AB + ACin + BCin$$

2. Quelles sont les entrées et sorties du *full adder*

Entrées :

- Les bits à additionner A et B
- Un bit représentant l'éventuelle retenue produite par une addition précédente Cin (*carry-in*)

Sorties :

- Le bit de somme S , addition binaire des bits d'entrée et de l'éventuelle retenue produite par une addition précédente
- Un bit représentant l'éventuelle retenue $Cout$ produite par cette addition (*carry-out*)

3. Complétez le fichier `full_adder.vhd` pour décrire en VHDL le *full adder*

Voir `src/full_adder.vhd`.

4. Ouvrez Vivado et créez un projet

Projet configuré suivant les instructions, pas de commentaire.

5. Complétez les commentaires dans le fichier de testbench `testbench_full_adder.vhd`.

Voir `src/testbench_full_adder.vhd`.

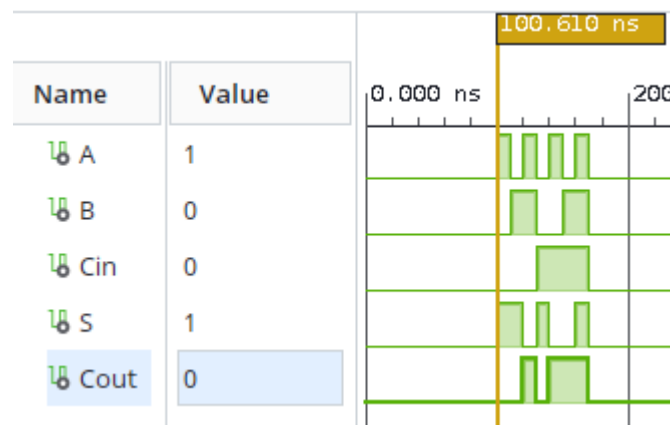
6. Sur Vivado, dans l'onglet « Sources », faites un clic droit sur « Simulation Sources » puis « Add Sources ».

Fichier source de simulation ajouté suivant les instructions, pas de commentaire.

7. Dans l'onglet « Flow Navigator », cliquez sur « Run Behavioral Simulation ».

Une fois la simulation exécutée, le chronogramme se trouve dans la fenêtre « Waveform ».

Tip : Une fois la simulation exécutée, cliquer sur « Zoom Fit » pour ajuster automatiquement la *timeline* du chronogramme.



On vérifie bien :

- S HIGH lorsque $A \oplus B \oplus Cin$ est vrai
- $Cout$ HIGH lorsque $AB + ACin + BCin$ est vrai

8. Modifiez le testbench pour ajouter des tests automatiques.

Voir `testbench_full_adder.vhdl`.

Les tests automatiques sont programmés avec des expressions VHDL `assert`.

9. Dans l'onglet « Flow Navigator », cliquez « Run Synthesis » pour exécuter une synthèse de votre architecture.

Voir rapport de synthèse `rpt/full_adder_utilization_synth.rpt`.

10. Déroulez le menu « Synthesis » et cliquez sur « Schematic » pour ouvrir la schématique. Où sont les portes logiques de l'architecture ?

Les portes logiques du *Full adder* sont programmées dans les cellules :

- **S_OBUF_inst_i1** : $S = A \oplus B \oplus Cin$
- **Cout_OBUF_inst_i1** : $Cout = AB + ACin + BCin$

